CSC1223
Data structures and Algorithms

Introduction

Some important points …

Need to run computer programs efficiently!

Computer program:

Accepts Input (Data)

Performs a Sequence of actions with the input

Generates Output (Data)

- Efficient Management of Data

  (Data Structures)

- Efficient Sequence of Actions

Algorithms

Algorithms

Sequence of actions a machine can follow

```
For j=1 to N
    print j
```

Efficient versus Inefficient algorithms

Design of Algorithms

You have a problem to solve

Design an efficient algorithm

Use good data structures

Show that your algorithm works!

Prove its correctness

Study the efficiency of your algorithm

Introduction to data structures

Data Structures

- A data structure is an arrangement of data in computer's memory.

- Algorithms manipulate the data in these structures in various ways.

Data Structures

- A data structure is a way to organize information to enable efficient computation over that information.
- A data structure supports certain operations, each with a
  - Meaning – what does the operation do
  - Performance – how efficient is the operation

Definition of data structures

- Many algorithms require a proper representation of data to achieve efficiency. This representation and the operations that are allowed for it are called data structures.
- Each data structure allows insertion, access, deletion and other operations.

Why we need data structures?

- Data structures allow us to achieve an important object oriented programming goal: component reuse.
- Once each data structure has been implemented, it can be used over and over again in various applications.

Data Structures and Algorithms

- The choice of a data structure and an algorithm can make the difference between a program running in a few seconds or many days.

Selecting a Data Structure

Select a data structure as follows:

1. Analyze the problem to determine the resource constraints a solution must meet.
2. Determine the basic operations that must be supported. Quantify the resource constraints for each operation.
3. Select the data structure that best meets these requirements.

Data Structure Philosophy

- Each data structure has costs and benefits.
- Rarely is one data structure better than another in all situations.
- A data structure requires:
  - space for each data item it stores,
  - time to perform each basic operation,
  - programming effort.

Read World Modeling

Some data structures directly model a real world situation.

- A queue for example, can model customers waiting at a bank.
- Can use graphs to represent airline routes between cities.

Some common data structures….

- Stacks
- Queues
- Lists
- Trees
- Graphs
- Tables

What is meant by Abstraction?

Omitting or hiding low-level details with a simpler, higher-level idea.

What is an **ADT**?
Abstract **D**ata **T**ype

- A way of looking at a data structure: focusing on what it does and ignoring how it does.
- An Abstract Data type is defined as a mathematical model of the data objects that make up a data type as well as the functions that operate on these objects. There are no standard conventions for defining them.

Abstract Data Type

- Fundamentally, a data type is a collection of values and a set of operations on these values. A useful tool for specifying the logical properties of a data type is the Abstract data Type .
- The term “Abstract Data Type” refers to the basic mathematical concept that defines the data type.

ADT …

- In defining an ADT as a mathematical concept, we are not concerned with space or time efficiency, these are implementation issues.
- There are a number of methods for specifying an ADT.

ADT ….

- Objects such as lists, sets, stacks, queues along with their operations can be viewed as Abstract Data Types.
- For the *set ADT*, we might have such operations as union, intersection, size and complement.
- ADT's definition does not mention how the set of operations is implemented.

ADT ….

- The basic idea is that the implementation of these operations is written once in the program and any other part of the program that needs to perform an operation on the ADT can do so by calling the appropriate function.
- If for some reason implementation details need to change, it should be easy to do so by merely changing the routines that perform the ADT operations.

Example - an ADT for strings

```
abstract typedef STRING;
abstract length(s)
STRING s;
postcondition length == len(s);

abstract STRING concat(s1,s2)
STRING s1, s2;
postcondition concat == s1+s2;
```

Data Structures vs. Algorithms

Data Structures

Represent objects of the ADT

Algorithms

Manipulate the data structures to implement the operations of the ADT

Study of Data Structures ..

The study of data structures involves two complementary goals.

1. Identify and develop useful mathematical entities and operations and to determine what class of problems can be solved using these entities and operations.
2. Determine representations for these abstract entities and to implement operations on these representations.